

Report Esonero 1: Web Scraper & Evaluator API

Andrea Cerone
2126029

Alessandro Cordischi
2019924

Andrea Giordano
2148782

Laboratorio di Ingegneria Informatica
Sapienza Università di Roma

1 Obiettivo del progetto

L'obiettivo del presente lavoro è lo sviluppo e la validazione di un sistema distribuito per l'Information Extraction. L'applicativo è in grado di eseguire operazioni di web scraping su domini eterogenei, pulire il codice HTML isolando il contenuto informativo rilevante e convertirlo in formato Markdown. Successivamente, il sistema integra un motore di valutazione per misurare l'accuratezza dell'estrazione confrontandola con un Gold Standard.

2 Organizzazione del codice

Il sistema è strutturato in due macro-componenti separati, Backend e Frontend, che comunicano tra loro. Questa divisione rende il progetto molto più ordinato, intuitivo e facile da mantenere: ogni modulo gestisce esclusivamente il proprio compito specifico.

2.1 Backend

Il core applicativo è sviluppato in Python utilizzando il framework FastAPI. La scelta è basata sul supporto nativo alla programmazione asincrona (asyncio), fondamentale per ottimizzare i tempi di attesa durante le richieste HTTP. Il server espone, sulla porta 8003, un'API completa documentata tramite Swagger UI, strutturata sui seguenti endpoint:

- **GET /domains:** Restituisce la lista dinamica dei domini supportati.
- **GET /parse:** Esegue lo scraping di un URL e restituisce il testo in Markdown.
- **POST /parse:** Accetta HTML *raw* ed esegue l'estrazione senza crawling.
- **GET /gold_standard:** Recupera la entry di validazione associata a un URL.

- **GET /full_gold_standard:** Restituisce l'intero dataset di validazione di un dominio.
- **POST /evaluate:** Confronta il testo parsato con il Gold Standard.
- **GET /full_gs_eval:** Esegue una valutazione massiva delegando il parsing offline alle istanze polimorfiche.

2.2 Frontend

L'interfaccia utente (Web UI) è sviluppata come un servizio indipendente ospitato sulla porta 8004. Utilizza il motore di templating **Jinja2** per generare dinamicamente le pagine e integra un menu a tendina popolato interrogando il backend (endpoint `/domains` e `/full_gold_standard`) per recuperare gli URL presenti nel Gold Standard. L'applicazione comunica con il backend in modo asincrono tramite la libreria **httpx**, gestendo due flussi distinti: l'analisi di un URL libero e il test di valutazione automatica. L'interfaccia offre un confronto visivo immediato tra il codice HTML sorgente, l'output Markdown dei parser e, nei test sul Gold Standard, il testo ideale con le relative metriche di precisione e recall.

3 Logica dei parser

La sfida dello scraping consiste nel separare il "segnale" dal "rumore" HTML. Per gestire questa complessità, abbiamo creato una struttura gerarchica che parte da **BaseWebParser**: questa classe centralizza le configurazioni del browser (Crawl4AI) e le funzioni di emergenza per il recupero dei titoli. Le classi figlie si occupano della pulizia specifica tramite BeautifulSoup4:

- **WikipediaParser:** Decompone elementi come `.infobox`, gallerie e note, mappando i tag `h2` e `h3` nei prefissi Markdown `##` e `###`.

- **ScaruffiParser:** Gestisce architetture "legacy" normalizzando l'albero HTML ed eliminando anomalie tramite Regex.
- **TravelStateGovParser:** Isola i *warning* ufficiali tramite selettori CSS (es. `.tsg-rwd-main-copy-body-frame`) ed escludendo header e footer.

Ogni parser implementa il metodo `parse_offline_html`, garantendo coerenza tra parsing online e offline.

4 Containerizzazione e Portabilità

L'applicazione è interamente gestita tramite Docker e `docker-compose.yaml`. Il sistema implementa una rete virtuale isolata che permette al frontend di comunicare con il backend tramite nomi di servizio (`http://backend:8003`). L'uso di immagini base `python:3.10-slim` riduce l'impronta del sistema, mentre le variabili d'ambiente permettono di configurare l'ambiente senza alterare il codice.

Un aspetto cruciale è la **gestione dei percorsi relativi** tramite `pathlib` e la macro `__file__`. Il calcolo dinamico dei path rende il software *plug-and-play* su qualsiasi ambiente Ubuntu immediatamente dopo l'istruzione `docker compose up --build`. È stata inoltre curata l'iniezione programmatica nel `sys.path` per evitare errori di importazione circolare.

5 Metriche di valutazione

Il modulo `evaluator.py` implementa un algoritmo *token-level*. Dati i testi privati del markup, i token vengono estratti tramite regex (ignorando case e punteggiatura). Si calcolano quindi:

$$Precision = \frac{|P \cap G|}{|P|} \quad (1)$$

$$Recall = \frac{|P \cap G|}{|G|} \quad (2)$$

$$F1-Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (3)$$

Questa valutazione penalizza sia l'over-fetching che l'under-fetching. Il backend implementa inoltre meccanismi di **fallback** per il titolo.

6 Valutazione parser

Il sistema è stato validato tramite il grader ufficiale, superando i test funzionali e di stabilità. Oltre alle metriche di accuratezza, l'applicativo è stato

testato su casi limite come input vuoti, URL irraggiungibili e HTML malformati, restituendo sempre risposte coerenti e codici di errore HTTP appropriati.

Come si evince dalla Tabella 1, i risultati mostrano metriche di eccellenza. Su Wikipedia e Scaruffi, la Precision vicina al 0.99 indica una pulizia chirurgica del rumore. Su *travel.state.gov*, il sistema raggiunge una Recall perfetta (1.000) e un F1-Score di 0.9909, confermando che i selettori mirati assicurano la completa estrazione delle informazioni critiche sulla sicurezza.

Dominio	Precision	Recall	F1-Score
en.wikipedia.org	0.9988	0.9227	0.9515
www.scaruffi.com	0.9911	0.9460	0.9645
travel.state.gov	0.9823	1.0000	0.9909

Table 1: Valutazione Media finale sul Gold Standard

7 Conclusione

Il progetto ha dimostrato come un'architettura modulare permetta di affrontare la complessità del web scraping in modo scalabile. L'integrazione tra un backend asincrono e parser specializzati ha portato a un sistema capace di estrarre informazioni con una precisione estremamente elevata. La scelta di utilizzare Docker garantisce la totale portabilità del sistema, rendendolo pronto per il deployment in qualsiasi ambiente. Il successo nei test ufficiali e il punteggio finale di **0.98/1.00** validano pienamente le scelte architetturali e metodologiche effettuate, lasciando aperta la strada per future estensioni a nuovi domini.